

Structs

A **Struct** (short for "structure") is a composite data type that allows you to group together variables of different types into a single unit. It is the Go equivalent of a "Class" in other languages, but simpler.

1. Why Use Structs?

Without structs, you would have to manage multiple independent variables (e.g., name1, address1, phone1). This is messy because:

- The programmer must manually remember which variables belong together.
- It's hard to pass a "person" to a function if their data is scattered across three variables.
- **The Solution:** A Struct aggregates these fields into one object, making the relationship between the data explicit and organized.

2. Declaring a Struct Type

To define a struct, you use the type and struct keywords. Each element inside is called a **Field**.

```
type Person struct {  
    name    string  
    address string  
    phone   string  
}
```

3. Creating and Initializing Structs

There are three common ways to bring a struct to life in memory:

- **Using var (Zero Value):** This creates a struct where every field is set to its "Zero Value" (empty strings for strings, 0 for ints).

```
var p1 Person // name, address, and phone are all ""
```

- **Using the new() function:** This allocates memory for the struct and returns a pointer to it. All fields are initialized to their zero values.

```
p1 := new(Person)
```

- **Using a Struct Literal:** This is the most common way to create and populate a struct at the same time.

```
p1 := Person{  
    name:    "Joe",  
    address: "123 Main St",  
    phone:   "555-0199", // Note: trailing comma is required in Go!  
}
```

4. Accessing Fields (Dot Notation)

Unlike arrays or maps that use square brackets [], Structs use **Dot Notation** to read or write data to specific fields.

- **Writing:** `p1.name = "Joe"`
- **Reading:** `fmt.Println(p1.address)`

Expert Takeaway:

You will find that Structs are the backbone of Go applications. When you're building a backend, you will define a Struct to match your database schema or your API's JSON response.

Pro Tip: In Go, if you start a field name with a Capital letter (e.g., `Name` instead of `name`), that field is **exported**, meaning other packages can see it. If you're building an API and your fields are lowercase, Go's JSON encoder won't be able to "see" them to turn them into **JSON**!

Always remember: **Upper Case = Public, Lower Case = Private.**